

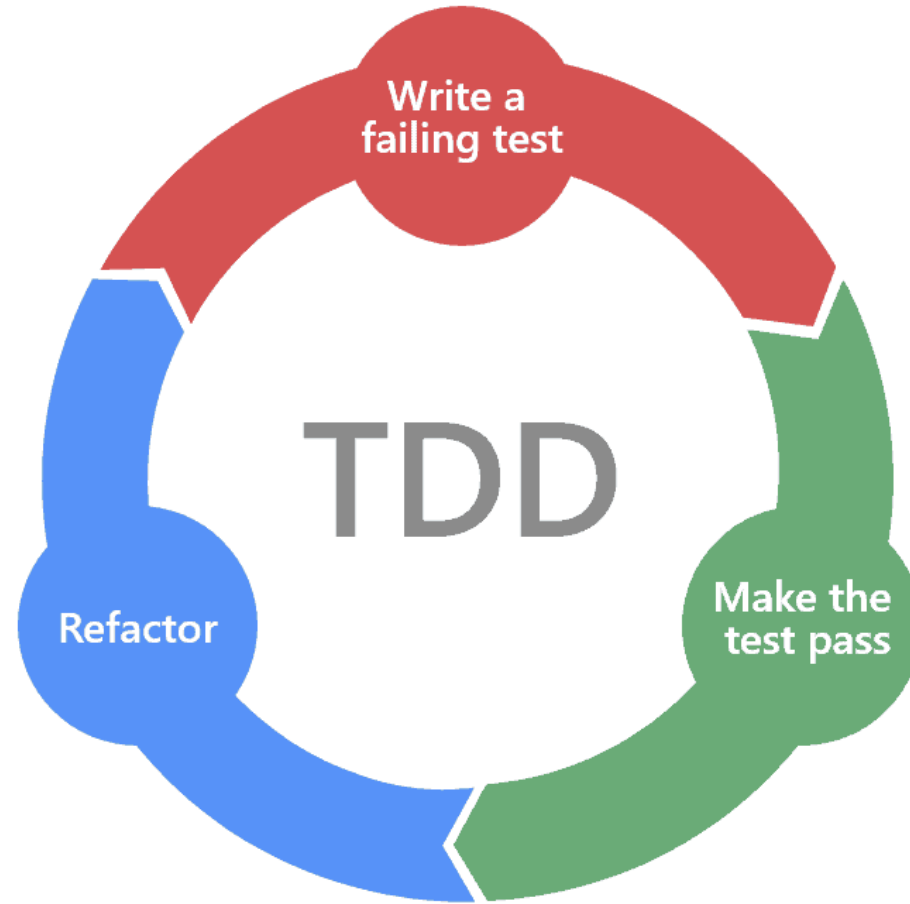


Full-stack software development

Testing with Jest

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Test Driven Development



Jest: JavaScript Testing Framework



- <https://jestjs.io/>
- <https://jestjs.io/docs/getting-started>
- <https://jestjs.io/docs/using-matchers>

Setting up a test environment

Install Jest	
Add Jest to your project	<code>npm install --save-dev jest</code>
Install Dependencies to let use Jest to test TS-projects • Test runner, test framework • Typescript preprocessor support for Jest • Type definition for Jest	<code>npm i -D jest ts-jest @types/jest</code>
Tell Jest that we want to use TS-Jest as a preprocessor => <code>jest.config.js</code> is created	<code>npx ts-jest config:init</code>
Add test command in <code>package.json</code>	<code>package.json:</code> <code> "scripts": {</code> <code> "test": "jest",</code> <code> "start": "nodemon -L app.ts --delay 3"</code> <code> }</code>
Write a test	
Run your test (runs all tests with name <code>*.test.ts</code>)	<code>npm test</code>

<https://levelup.gitconnected.com/tdd-with-typescript-and-jest-starter-project-cca94fd089f5>

Simple Unit Testing

```
20 ✓ test(`given: valid value for name and password; when: user is created; then: empty fields are undefined`, () => {  
21   const user: User = new User({ password: validPassword, username: validUsername });  
22   expect(user.username).toEqual(validUsername);  
23   expect(user.getId()).toBeUndefined();  
24   expect(user.getPassword()).toEqual(validPassword);  
25   expect(user.getProfile()).toBeUndefined();  
26 })
```

```
28 test(`given: usernameTooShort but other fields correct; when: user is created; then: error is thrown`, () => {  
29   expect(() => new User({ id: validId, password: validPassword, username: usernameTooShort, profile: validProfile }))  
30     .toThrow('Username is too short');  
31 }  
32 }
```

Syntax: *test(name,function)*

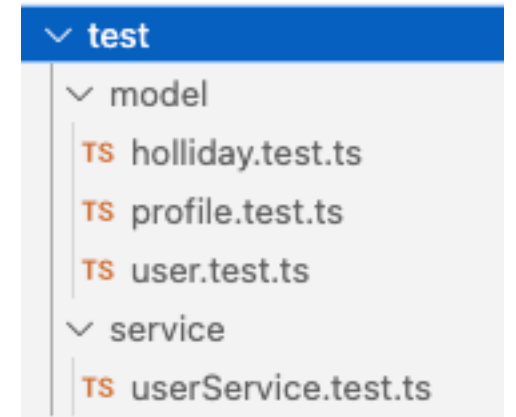
Keyword

Name of the test

Anonymous function to be executed

Naming Convention

- *.test.ts, e.g. “user.test.js”
- Create a folder “test” and structure it in the same way as your source code
- Name of test function: use given/when/then structure



Matchers

See <https://jestjs.io/docs/expect> for full list

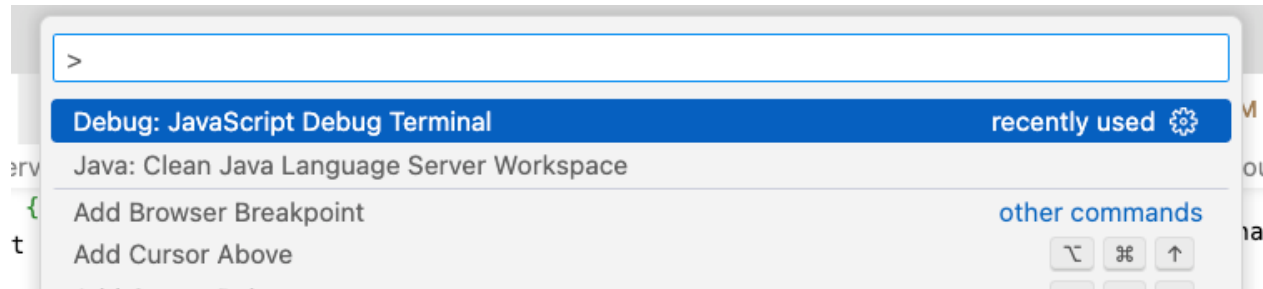
- `.toBe()`: equality
- `.toEqual()`: equality of the value of an object (checks every field)
- `.toBeNull()`, `toBeUndefined()`
- `.toBeTruthy()`, `toBeFalsy()`
- `.toMatch()`: check string against regular expression
- `.toContain()`: check if an array contains a particular item
- `.toThrow()`: check whether a function throws an error

Setup and Teardown

- Repeating Setup: `beforeEach()` and `afterEach()`
- One-Time Setup: `beforeAll()` and `afterAll()`

Run Test in Debug Mode

- Open JavaScript Debug Terminal
in VS Code: F1 > Debug: Javascript Debug Terminal



- Run tests in that terminal in a Watch mode `npm test --watch`
- Set a break-point in your file

References

- <https://jestjs.io/docs/getting-started>
- <https://jestjs.io/docs/api>